Design Patterns

Reusable Design Decisions for Evolving Financial Systems

FinTech 512 – Software Engineering

Duke

Where We Are in the Course

Object-Oriented Design
Making design decisions explicit

Design Principles (SOLID)
Evaluating design decisions

Smells & Refactoring
Improving decisions without changing behavior

Design Patterns
Reusing proven decisions

Duke

Review: Object-oriented Programming

- Everything is an object
- Program is a bunch of objects telling each other what to do (sending messages)
- Each object has its own memory/state made up of other objects
- Every object has a type
- All objects of a particular type can receive the same messages

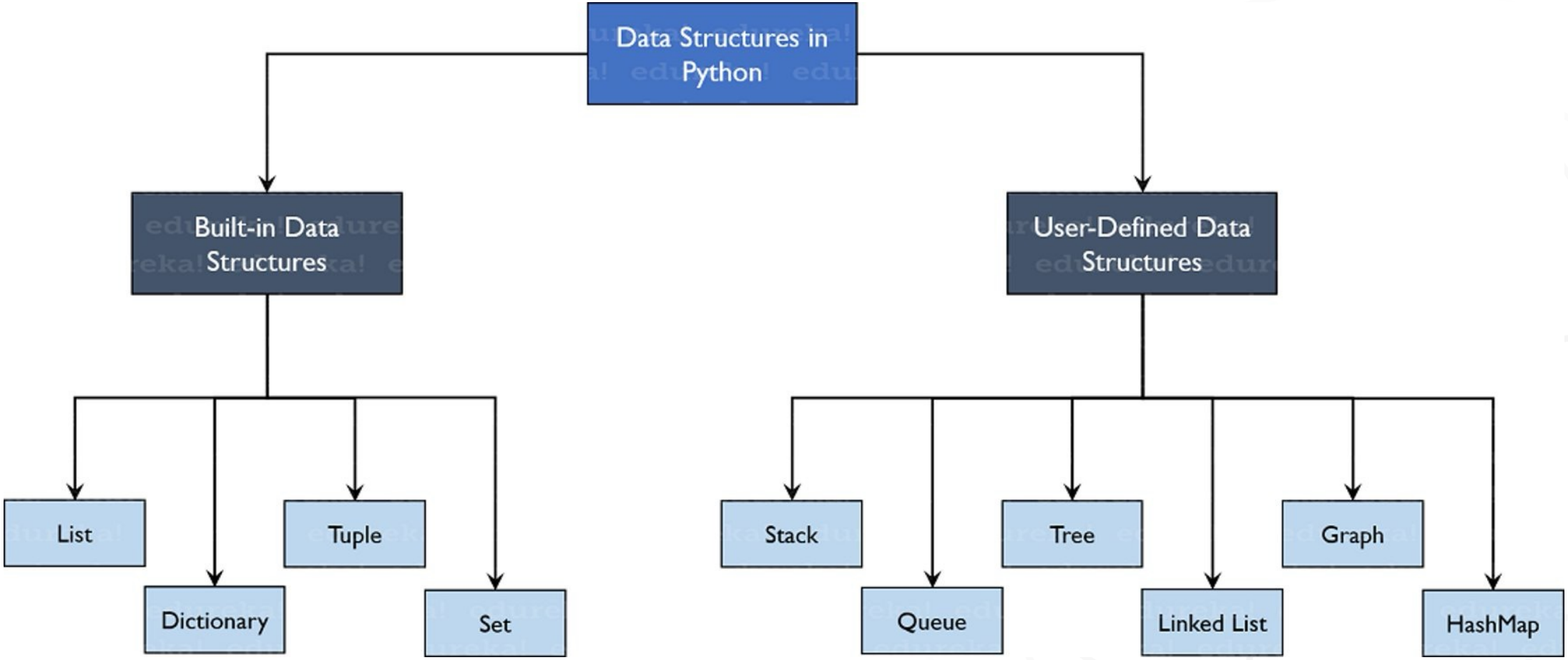
Object: state, behavior, and identity

Class:

- Blueprint from which individual objects are created
- Abstract data type

Duke

Collections / Data Structures

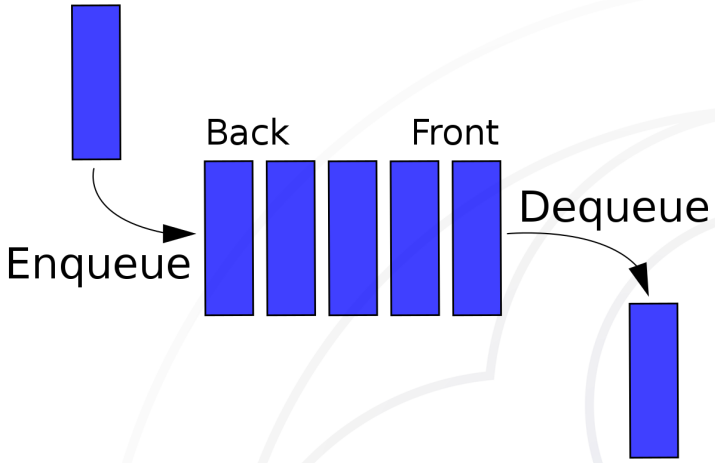


https://www.edureka.co/blog/data-structures-in-python

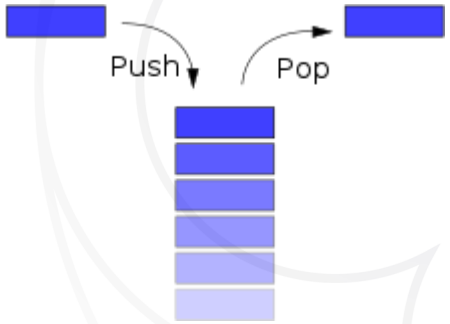
Duke

Lists: Specialized Versions

- Queue
  - enqueue (add, put)
  - dequeue (remove, pull, poll)
  - peek



- Stack
  - push
  - pop
  - peek



Duke

Problem: incompatible interfaces

- Newly received client source code has dozens/hundreds of references to a stack class with push()/pop() methods, but your codebase only has the Python list class, with append() and pop().
- Solution: Add a class that converts from the interface you have (append/pop) to the interface you need (push/pop)

List supports many operations:

- append
- pop
- insert
- index access

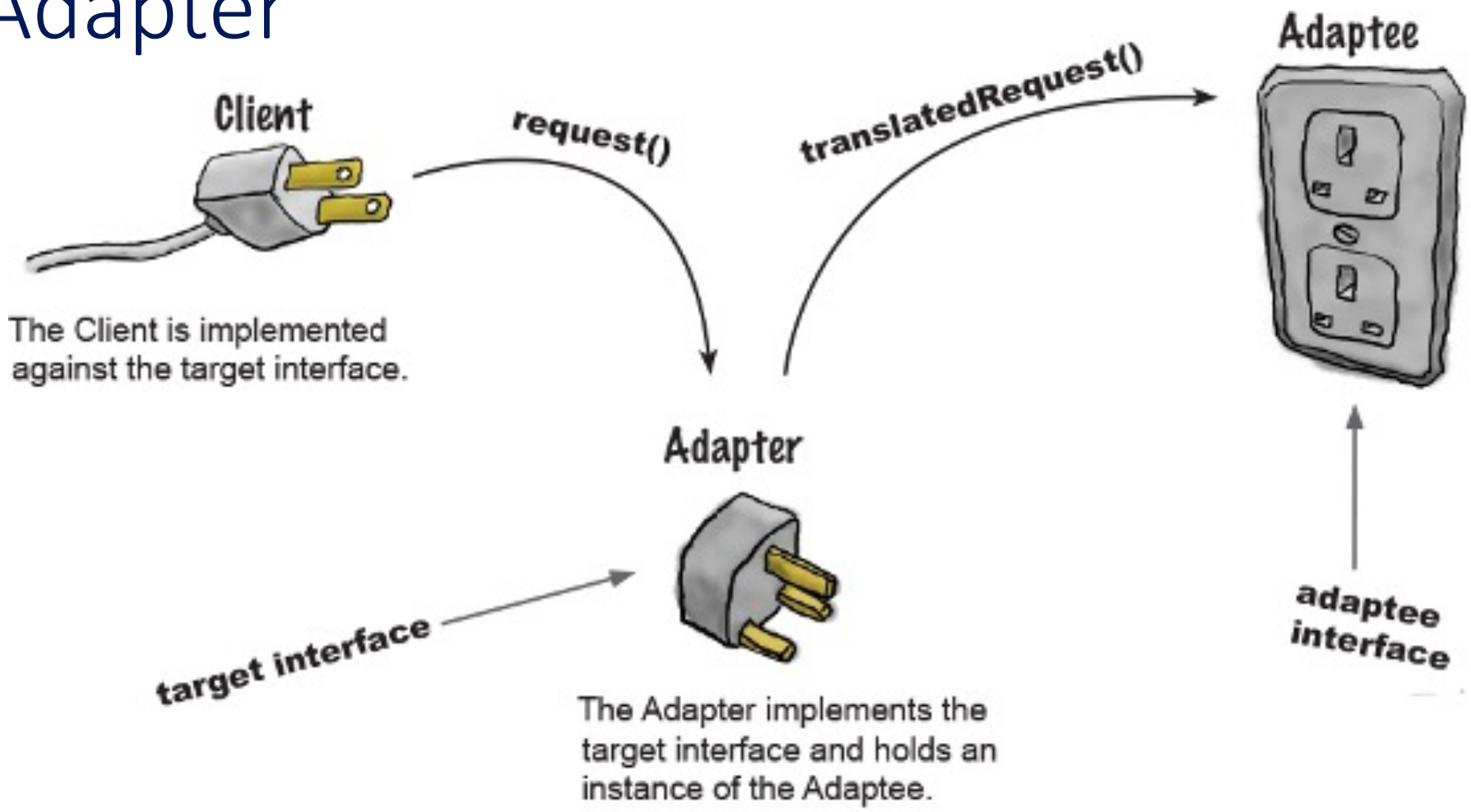
But a **stack** or **queue**:

- Uses only a subset
- Renames operations for intent
- Restricts how data flows/is accessed

Same storage, different contract

Duke

Adapter



"Head First Design Patterns", Freeman, Robson

Stack Implementation (Python)

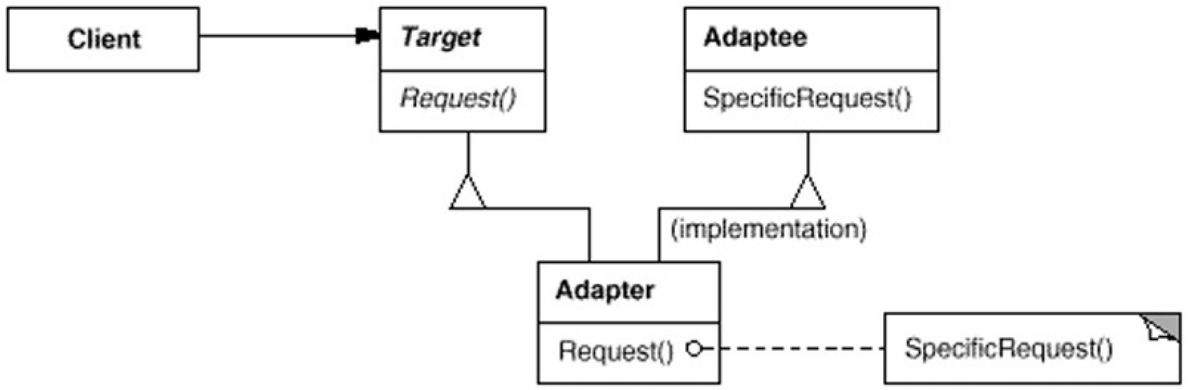
```
class Stack:
    def __init__(self):
        self.__data = []
    def push(self,item):
        self.__data.append(item)
    def pop(self):
        return self.__data.pop()
    def peek(self,index):
        return self.__data[index]
    def size(self):
        return len(self.__data)
    def __len__(self):
        return len(self.__data)
```

Duke

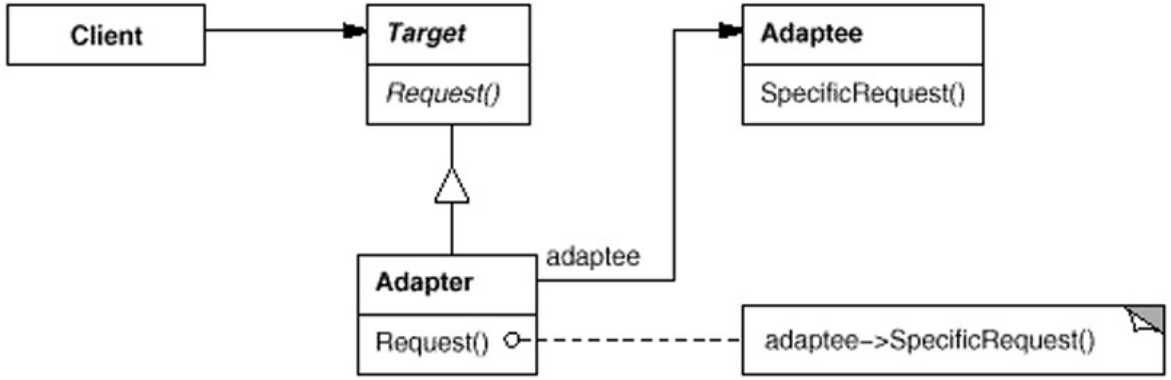
Adapter

- Target
  - defines the interface you need to implement
- Client
  - collaborates with objects conforming to the Target Interface
- Adaptee
  - Defines an existing interface that needs adapting
- Adapter
  - Adapts the interface of Adaptee to the Target Interface

"class-based" adapter



"object-based" adapter



Duke

Stack Implementation – class based

```
class StackClassAdapter(list):
    def push(self, item):
        self.append(item)

    def pop(self):
        return super().pop()

    def peek(self):
        return self[-1]

    def is_empty(self):
        return len(self) == 0
```

Duke

Adapter: Tradeoffs

Class Adapter (Inheritance-based)

What you gain

- Can **override or reuse** Adaptee behavior
- Fewer objects → slightly simpler structure

What you give up

- Tightly coupled to a **specific Adaptee**
- Cannot adapt multiple Adaptees
- Breaks if Adaptee hierarchy changes

Object Adapter (Composition-based)

- **What you gain**
  - Can adapt **many different Adaptees**
  - Adaptee can be **swapped at runtime**
  - Lower coupling, better isolation
- **What you give up**
  - Cannot easily override Adaptee behavior
  - One extra level of indirection

"Which version would you expect to see in a financial system that integrates with third-party vendors — and why?"

Duke

What is a Design Pattern?

- Describes a **recurring design problem**
- Applies in a **specific context**
- Defines a **structure of collaborating roles**
- Comes with **known tradeoffs**

Is

- Problem
- Context
- Structure
- Consequences

Is Not

- Code template
- Architecture
- Proof of good design

Duke

Why Design Patterns Exists

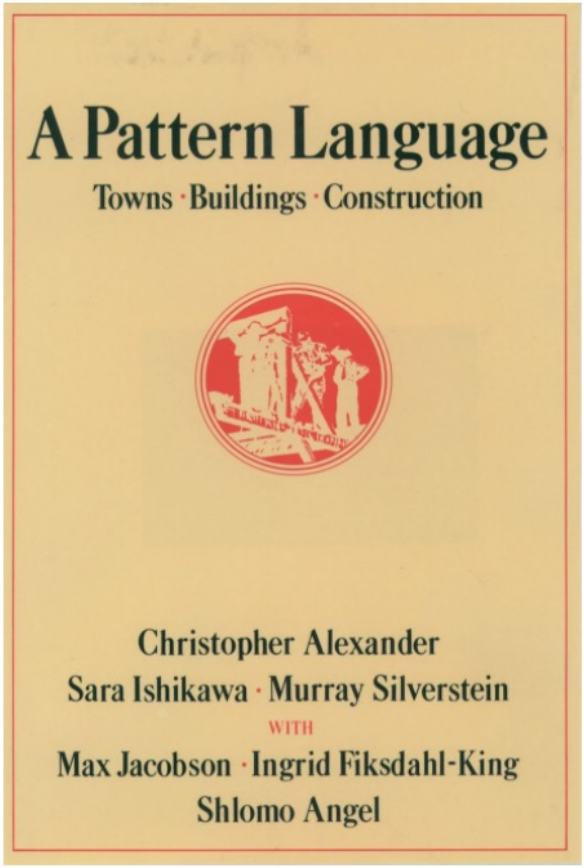
- Engineers encounter the **same design problems** repeatedly
- Teams independently rediscover similar solutions
- Experience gets lost without shared names
- Patterns preserve **hard-won design knowledge**

Patterns encode experience — not invention

Duke

A Pattern Language

- Concept introduced by architect **Christopher Alexander**
- Patterns describe:
  - Problems that recur
  - Solutions that work repeatedly
  - Contexts where they apply
- Software design patterns apply the same idea:
  - Reusable solutions
  - Adapted to context
  - Never copied verbatim



Patterns as a Design Vocabulary

Patterns provide a **shared language** for design:

- Enable precise communication
- Improve design reviews
- Shift discussions from **opinions to decisions**

Patterns let teams reason together about design

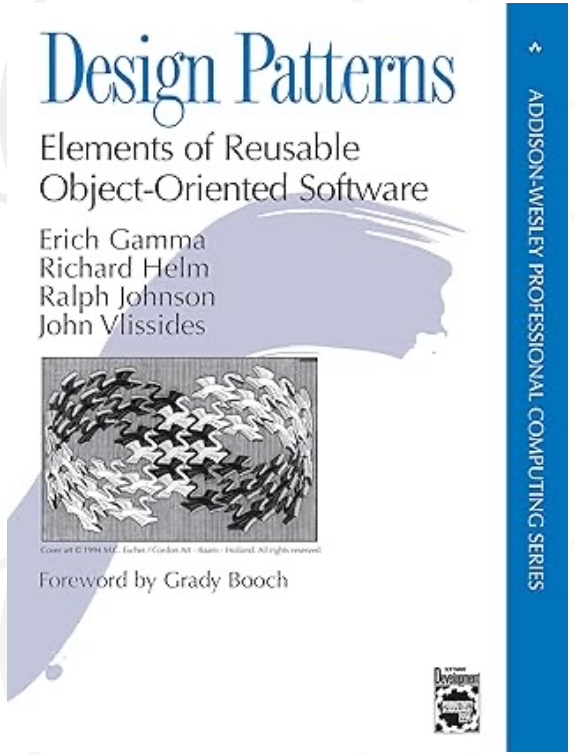
Elements of a Design Pattern (GoF)

Name
A shared term for communication

Problem
When the pattern applies

Solution
Roles, responsibilities, and collaborations

Consequences
Benefits **and** costs of using the pattern



Pattern Families

Design patterns are grouped by the **kind of decision they address**:

- **Creational**
  - *How are objects created?*
  - Who controls creation and configuration?
- **Structural**
  - *How are responsibilities composed?*
  - How are objects connected and wrapped?
- **Behavioral**
  - *Where do decisions live?*
  - How does control and behavior vary?

Creational Patterns — Overview

Address **how objects are created** and **who controls that creation**.

In an **online brokerage**, creation decisions affect:

- Consistency of system state
- Correctness under scale
- Testability and change safety

Common questions:

- Should there be **one shared instance** or many?
- Who decides **which implementation** to use?
- How do we prevent clients from depending on concrete classes?

Singleton Pattern

Problem

In an online brokerage:

- Multiple components need access to the **same exchange connection**
- Independent instances risk:
  - Inconsistent market state
  - Duplicate external connections
  - Conflicting actions

Solution

- Restrict instantiation so **only one instance exists**
- Provide a **single, well-known access point**
- Ensure all clients coordinate through that instance

Consequences

Benefits

- Consistent system-wide view
- Controlled access to shared resources

Costs

- Hidden global dependency
- Harder to test and replace
- Concurrency and lifecycle risks

Duke

Singleton: Python example

```python
class Singleton:
    _instance = None

   def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance


# Usage example
a = Singleton()
b = Singleton()

print(a is b) # True
```

Note: Assumes single-threaded execution when the Singleton is first instantiated.

Duke

Factory Pattern

Problem

In an online brokerage:

- Clients need to create **different kinds of orders** (Market, Limit, Stop, etc.)
- Creation rules vary by:
  - Order type
  - Asset class (stock, option, crypto)
  - Venue / routing policy
  - Compliance / risk constraints
- If clients construct orders directly:
  - They depend on concrete classes
  - Rules get duplicated and inconsistently applied
  - Adding a new order type breaks many callers

Solution

Create a dedicated component that:

- Encapsulates **object creation**
- Returns a **common interface** (Order)
- Centralizes validation / configuration / defaults

Consequences

Benefits

- Looser coupling (clients depend on abstractions)
- Centralized creation rules and invariants
- Easier to add new order types or policies

Costs

- Extra indirection / one more component
- Risk of a “god factory” if it accumulates too many responsibilities
- Must design stable interfaces for products

Duke

Online Brokerage: Order Creation Pressure

Clients want to do this:

```
place_order(symbol="AAPL", qty=100, type="limit", limit_price=192.50)
```

But creation must decide:

- Which Order subtype?
- Which defaults (TIF, routing, disclosures)?
- Which validations apply?
- Which risk/compliance checks must run *before* submission?

Without a Factory

- UI, API, and services each re-implement creation rules
- Invariants drift → inconsistent behavior

With a Factory

- One creation entrypoint
- One set of defaults and validations
- One place to review changes

Duke

```python
from dataclasses import dataclass
from abc import ABC, abstractmethod

class Order(ABC):
    @abstractmethod
    def describe(self) -> str: ...

@dataclass(frozen=True)
class MarketOrder(Order):
    symbol: str
    qty: int

    def describe(self) -> str:
        return f"MARKET {self.qty} {self.symbol}"

@dataclass(frozen=True)
class LimitOrder(Order):
    symbol: str
    qty: int
    limit_price: float

    def describe(self) -> str:
        return f"LIMIT {self.qty} {self.symbol} @ {self.limit_price}"
```

Duke

```python
class OrderFactory:
    @staticmethod
    def create(order_type: str, **kwargs) -> Order:
        t = order_type.lower()
        if t == "market":
            return MarketOrder(symbol=kwargs["symbol"], qty=kwargs["qty"])

        if t == "limit":
            return LimitOrder(symbol=kwargs["symbol"], qty=kwargs["qty"],
                              limit_price=kwargs["limit_price"])

        raise ValueError(f"Unknown order type: {order_type}")
```

Duke

Structural Patterns — Overview

Address **how objects and classes are composed**.

They help when you need to:

- Integrate with **legacy or vendor interfaces**
- Add behavior **without rewriting** existing code
- Provide a simpler interface to a complex subsystem
- Protect systems from change by creating **stable boundaries**

Common structural patterns:

- **Adapter** — match incompatible interfaces
- **Decorator** — add responsibilities dynamically
- **Facade** — simplify access to a subsystem

Structural patterns change the shape of a system, not its purpose

Duke

Facade Pattern

Problem

In an online brokerage:

- Clients must interact with **multiple subsystems**:
  - Market data
  - Order placement
  - Account balances
  - Risk checks
- Direct access exposes:
  - Too many details
  - Ordering dependencies
  - Tight coupling to internals
- Small changes in one subsystem ripple everywhere

Solution

Introduce a **Facade** that:

- Provides a **simplified, high-level interface**
- Coordinates calls to multiple subsystems
- Shields clients from internal complexity

Consequences

Benefits

- Reduced coupling
- Easier client usage
- Internals can evolve independently

Costs

- Facade can become too large if poorly scoped
- May hide useful low-level functionality

Duke

```python
class BrokerageService:
def __init__(self, market_data, risk_engine, order_router, accounts):
    self.market_data = market_data
    self.risk_engine = risk_engine
    self.order_router = order_router
    self.accounts = accounts

def place_order(self, account_id, order):
    balance = self.accounts.get_balance(account_id)
    price = self.market_data.get_price(order.symbol)

    self.risk_engine.check(order, balance, price)
    self.order_router.submit(order)

def get_account_summary(self, account_id):
    return {
        "balance": self.accounts.get_balance(account_id),
        "positions": self.accounts.get_positions(account_id)
    }
```

Duke

Behavioral Patterns – Overview

Address **how decisions are made and coordinated**.

They help when:

- Behavior varies by **policy or context**
- Decisions must change **without changing clients**
- Objects must **react to events**
- Control flow should be explicit and testable

Common behavioral patterns:

- **Strategy** — encapsulate and swap decision logic
- **Observer** — react to state changes
- **Iterator** — traverse collections without exposing structure

Behavioral patterns decide *where* behavior lives

Duke

Iterator Pattern

Problem

Collections often store data in a specific internal representation (list, tree, graph, DB cursor).
Clients still need to **traverse elements** without:

- Exposing internal structure
- Depending on representation details
- Breaking when the structure changes

Solution

Provide an **Iterator object** that:

- Accesses elements sequentially
- Maintains traversal state
- Offers a standard traversal interface

Consequences

Benefits

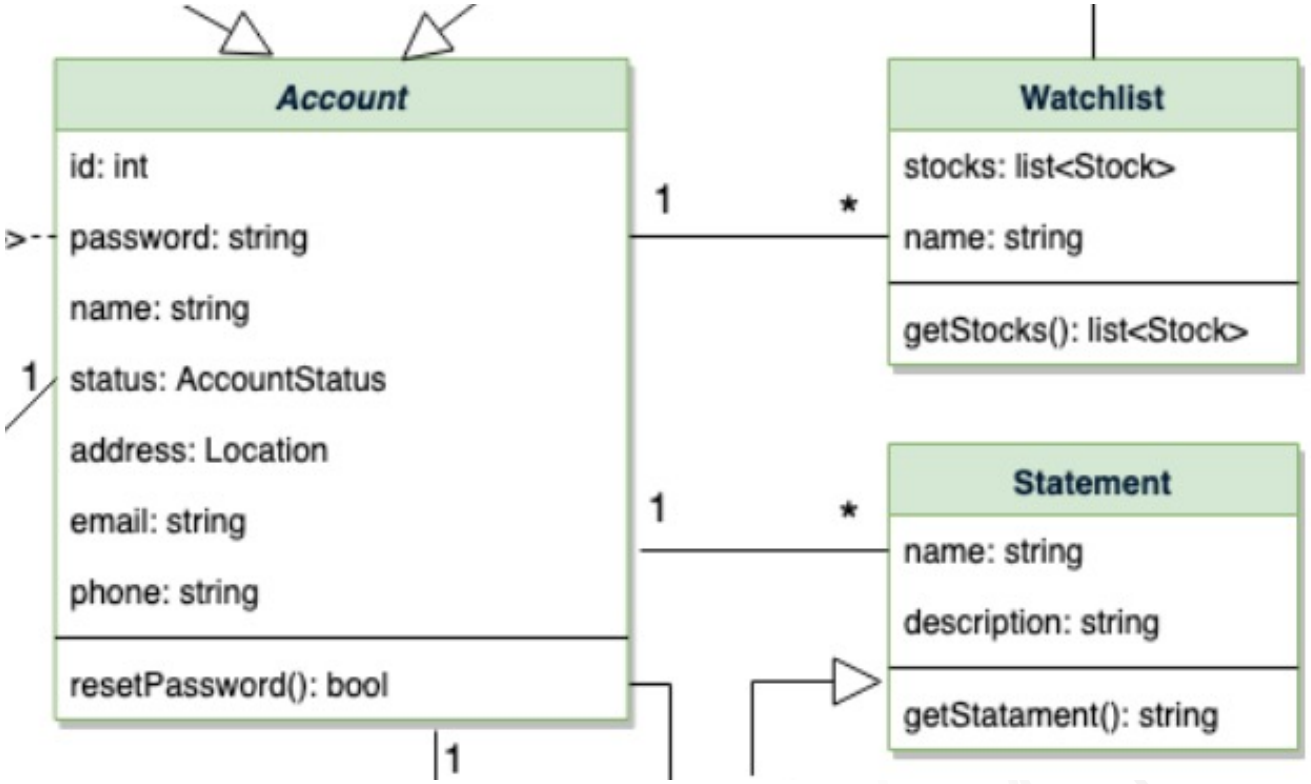
- Encapsulation: clients don’t depend on internal representation
- Multiple traversal strategies (forward, filtered, sorted)
- Cleaner client code

Costs

- Additional objects / indirection
- Must define behavior for concurrent modification

Duke

Watchlist - Good Design?



Duke

Solution A: GoF Pattern – Separate Iterator Class

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Stock:
    symbol
    …

class WatchlistIterator:
    def __init__(self, stocks):
        self._stocks = stocks
        self._index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self._index >= len(self._stocks):
            raise StopIteration

        stock = self._stocks[self._index]
        self._index += 1
        return stock
```

```python
class Watchlist:
    def __init__(self, name):
        self.name = name
        self._stocks = []

    def add(self, stock):
        self._stocks.append(stock)

    def __iter__(self):
        return WatchlistIterator(self._stocks)
```

```python
wl = Watchlist("Tech")
wl.add(Stock("AAPL", "NASDAQ"))
wl.add(Stock("MSFT", "NASDAQ"))

for stock in wl:
    print(stock)
```

Duke

Solution B: Make Watchlist Iterable

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Stock:
    symbol: str
    …

class Watchlist:
    def __init__(self, name, stocks):
        self.name = name
        self._stocks: list[Stock] = list(stocks) if stocks is not None else []

    def add(self, stock: Stock) -> None:
        self._stocks.append(stock)

    def remove(self, symbol: str) -> None:
        self._stocks = [s for s in self._stocks if s.symbol != symbol]

    def __iter__(self) -> Iterator[Stock]:
        # Returning an iterator over the internal list
        return iter(self._stocks)

    def __len__(self) -> int:
        return len(self._stocks)

    def __repr__(self) -> str:
        return f"Watchlist(name={self.name!r}, stocks={len(self._stocks)})"
```

Duke

Observer Pattern

Problem

In many systems:

- One object's state changes
- Multiple other components must react
- The number and type of reactions may change over time

If observers are wired directly:

- The subject becomes tightly coupled
- Adding a new reaction requires modifying existing code

Solution

Define a **one-to-many dependency** where:

- Observers register interest
- The subject notifies observers when state changes
- Observers react independently

Consequences

Benefits

- Loose coupling between subject and observers
- New reactions added without changing the subject
- Clear event-driven structure

Costs

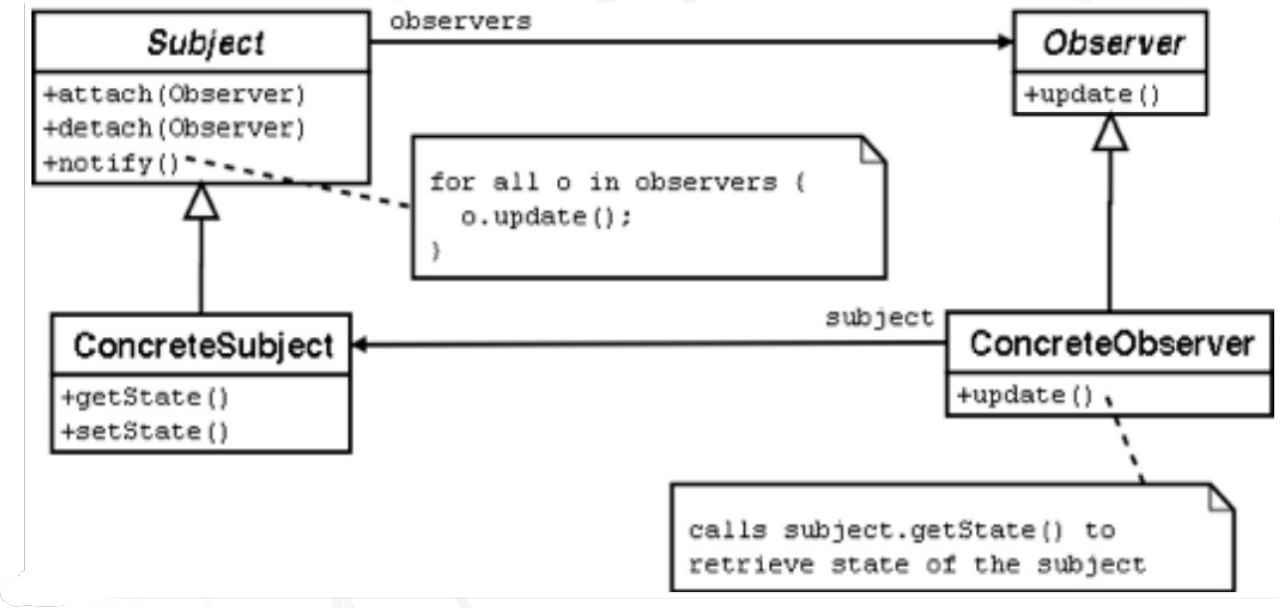
- Hidden control flow
- Harder debugging and reasoning
- Ordering and timing concerns

Duke

Observer – GoF Pattern

Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically

- **Subject:** knows its observers. Provides an interface to add/remove observers
- **Observer:** interface for objects to implement when they should be notified of changes
- **ConcreteSubject:** stores state of interests to observers
- **ConcreteObserver:** performs some appropriate action when subject's state changes



Duke

Online Brokerage Example: Market Data

When a stock price changes:

- Trading algorithms may react
- Risk monitors may recompute exposure
- User interfaces may refresh
- Alerts may be triggered

Without Observer:

- Market data code must know *all* consumers

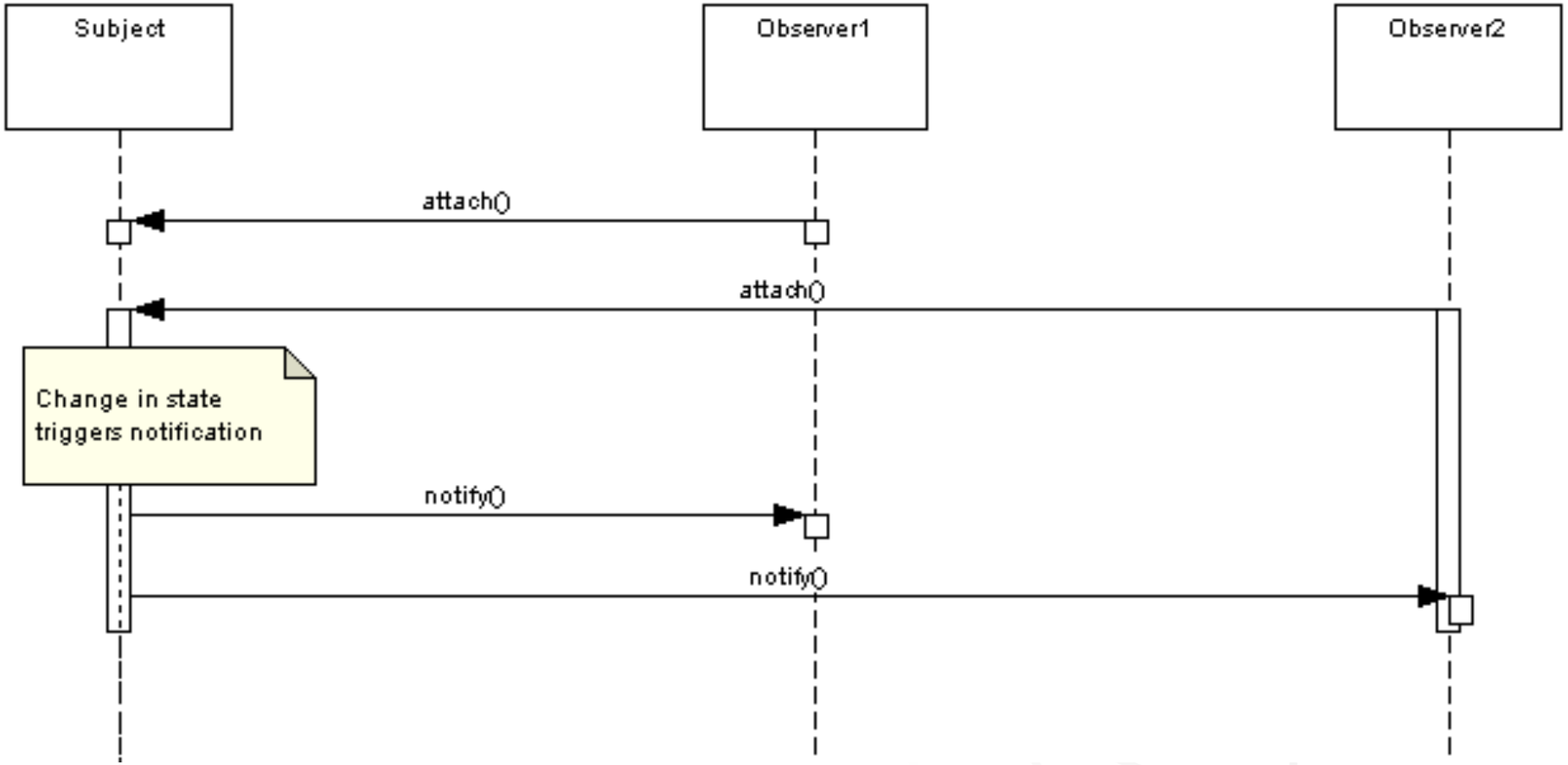
With Observer:

- Market data publishes updates
- Consumers subscribe independently

Market data changes once; reactions fan out

Duke

Observer Sequence Diagram



https://howtodoinjava.com/design-patterns/behavioral/observer-design-pattern/

Duke

StockExchange: Observer

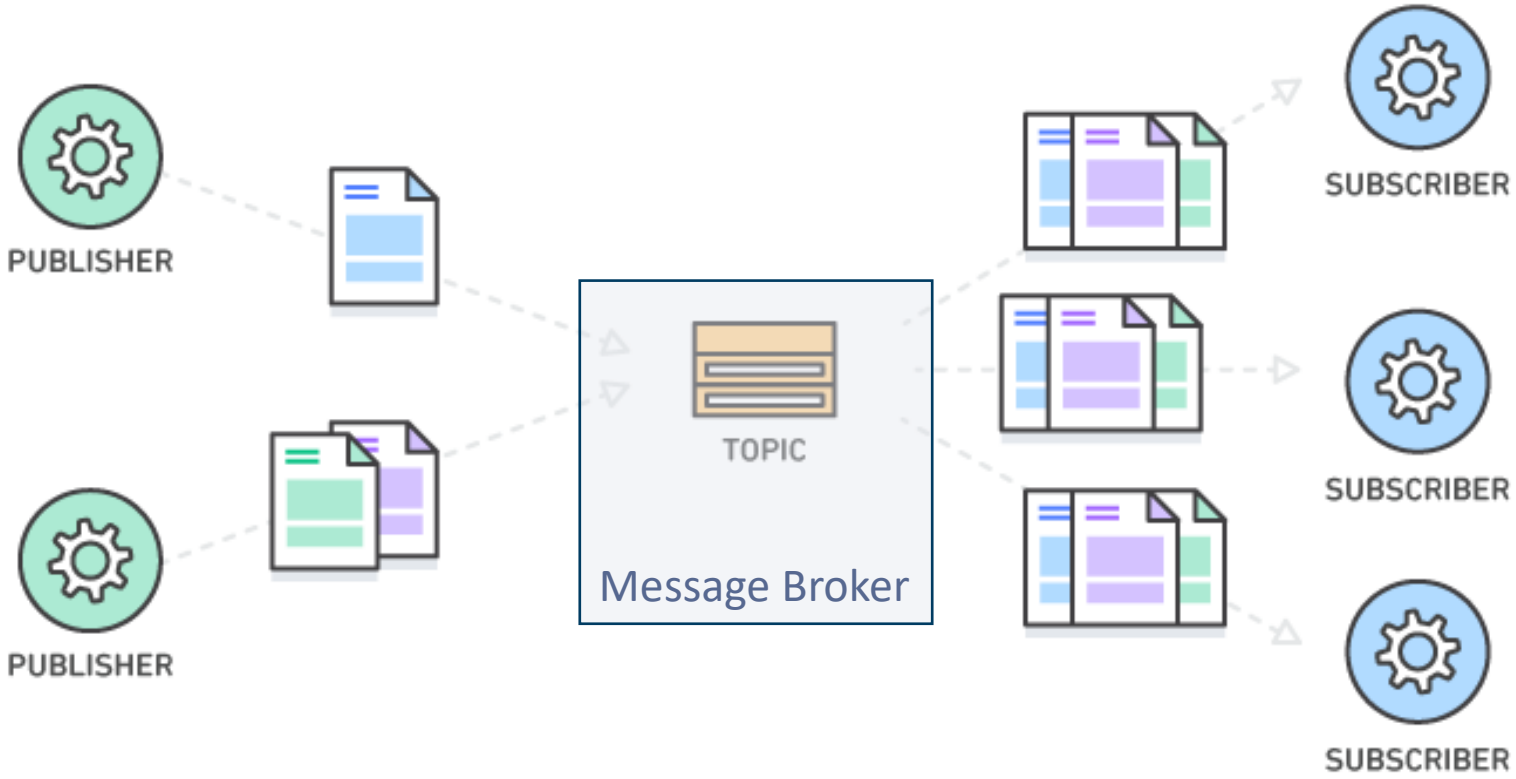
```python
class StockExchange:
  def __init__(self):
    self._observers = [] # List of observers
    self._stocks = {}    # Track stock prices

def add_observer(self, observer):
  """Attach an observer."""
  self._observers.append(observer)

def remove_observer(self, observer):
  """Detach an observer."""
  self._observers.remove(observer)

def notify_observers(self, stock, price):
  """Notify all observers of a stock price change."""
  for observer in self._observers:
    observer.update(stock, price)

def set_stock_price(self, stock, price):
  """Update stock price and notify observers."""
  self._stocks[stock] = price
  self.notify_observers(stock, price)
```

```python
from abc import ABC, abstractmethod
# Observer Interface
class Observer(ABC):
    @abstractmethod
    def update(self, stock, price):
        pass

# Concrete Observer Class
class Investor(Observer):
  def __init__(self, name):
    self.name = name

  def update(self, stock, price):
    """Receive stock price updates."""
    print(f"{self.name} received update:
          {stock} is now ${price}")
```

Pretend this is one line

```python
# Usage
if __name__ == "__main__":
    exchange = StockExchange()

    investor1 = Investor("Alice")
    investor2 = Investor("Bob")

    exchange.add_observer(investor1)
    exchange.add_observer(investor2)

    exchange.set_stock_price("AAPL", 150)
    exchange.set_stock_price("GOOGL", 2800)

    exchange.remove_observer(investor1)
    exchange.set_stock_price("TSLA", 750)
```

Publisher – Subscribe Pattern



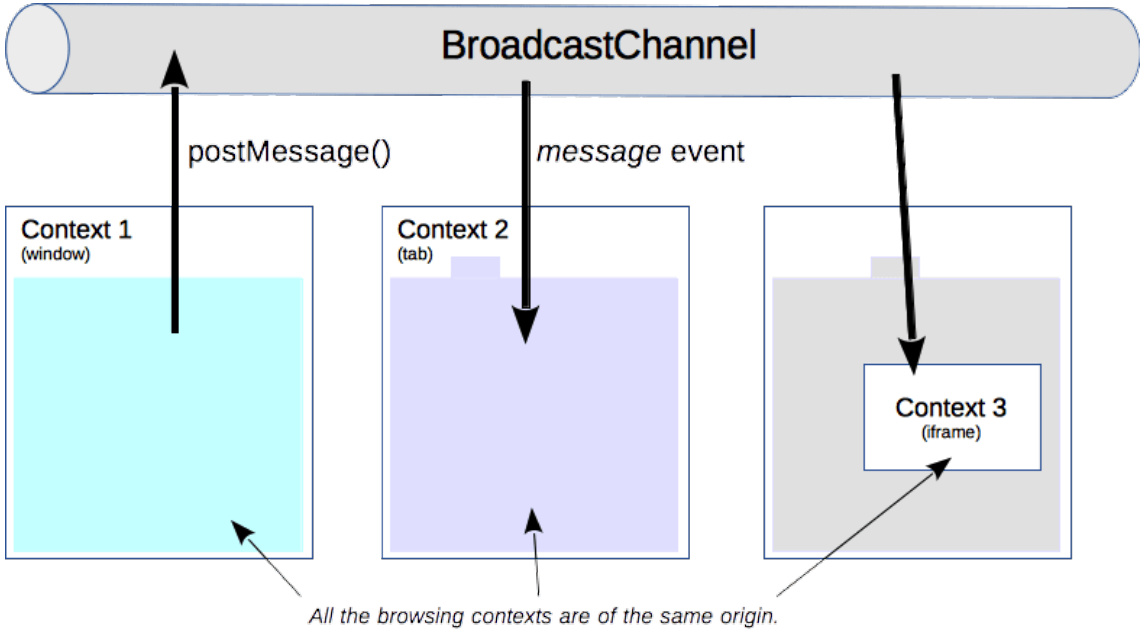
https://aws.amazon.com/pub-sub-messaging/

Duke

Pub-Sub for Browsers

Broadcast Channel API:

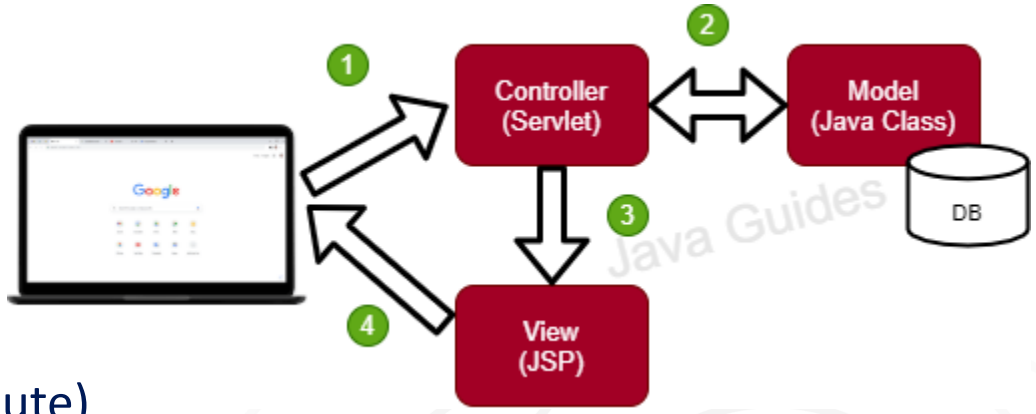
https://developer.mozilla.org/en-US/docs/Web/API/Broadcast_Channel_API




```
// Connection to a broadcast channel
const bc = new BroadcastChannel('test_channel');

// Example of sending of a very simple message
bc.postMessage('This is a test message.');

// A handler that only logs the event to the console:
bc.onmessage = function (ev) { console.log(ev); }
```

Duke

Model View Controller



1. Browser sends an HTTP request to a controller (route)
2. Controller/route validates request, interacts with model. Model may interact with a database or other sources
3. Controller render_template to produce the page
4. Template produces the web page to display on the browser

Model: business logic. System state. Independent of the presentation

Controller: interface between the model and view. Receives requests, validates, and processes

View: represents the output of the application, usually some UI

What about single page web applications? What's the view?

https://www.javaguides.net/2020/01/jsp-servlet-mvc-example-with-database.html Note: don't follow their example for the database connection

Duke

Simple Flask Application

flask_app/app.py

```python
import datetime
from flask import Flask, render_template

app = Flask(__name__)


@app.route('/')
def hello():
    return render_template('index.html', utc_dt=datetime.datetime.utcnow())
```

flask_app/templates/index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>FlaskApp</title>
</head>
<body>
    <h1>Hello World!</h1>
    <h2>Welcome to FlaskApp!</h2>
    <h3>{{ utc_dt }}</h3>
</body>
</html>
```

Strategy Pattern

Problem

A system needs to perform an operation, but:

- There are **multiple valid algorithms/policies**
- The choice may vary by **context**
- Requirements evolve over time
- If logic is embedded in conditionals:
  - Code becomes brittle
  - Adding a new policy requires modifying existing code

Solution

Define a family of algorithms as **strategies**:

- A common **Strategy interface**
- Multiple **ConcreteStrategy** implementations
- A **Context** that uses a Strategy

Consequences

Benefits

- Policies are isolated, testable, and replaceable
- Supports OCP (add new strategies without editing context)
- Reduces long conditional logic

Costs

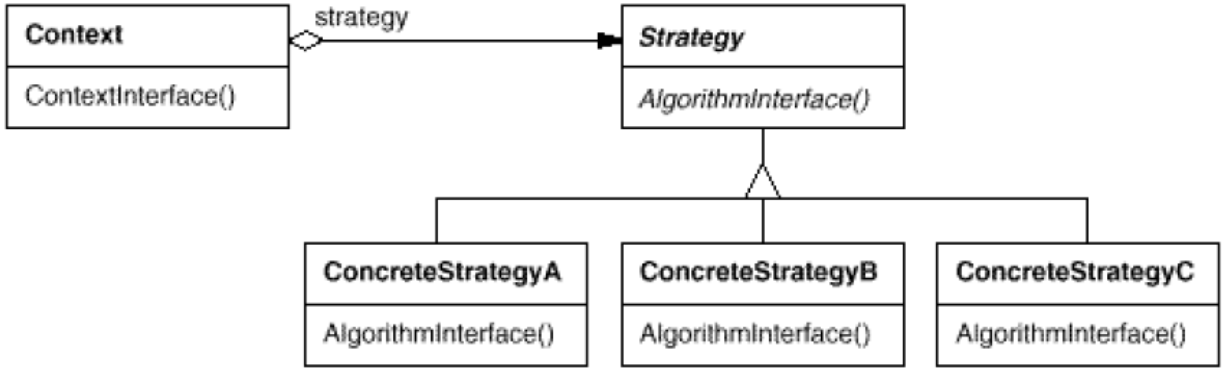
- More classes/objects
- Strategy selection becomes its own decision (who chooses?)

Duke

Strategy

Define a family of algorithms, encapsulate each one, and make them interchangeable. Algorithms can vary independently from clients that use them

Strategy: interface common for all support algorithms



ConcreteStrategy: implements a particular algorithm using the Strategy interface

Context: configured with a ConcreteStrategy object, may allow that object to access its state.

Duke

Online Brokerage Example: Fee Calculation

Order fees may depend on:

- Account tier (standard vs premium)
- Asset class (equities vs options)
- Volume discounts
- Promotions / regulatory fees

Bad design:

- if/elif fee logic scattered across services

Strategy-based design:

- FeeStrategy defines the fee contract
- Brokerage service uses the chosen strategy
- New fee policies added safely

Fee rules are policies — policies should be swappable

Duke

Strategy: Python Example (Fee Strategies)

```python
from abc import ABC, abstractmethod

class FeeStrategy(ABC):
    @abstractmethod
    def fee(self, order) -> float:
        pass

class FlatFee(FeeStrategy):
    def __init__(self, amount: float):
        self.amount = amount

    def fee(self, order) -> float:
        return self.amount

class PercentageFee(FeeStrategy):
    def __init__(self, rate: float):
        self.rate = rate

    def fee(self, order) -> float:
        return order.notional * self.rate
```

```python
class BrokerageService:
    def __init__(self, fee_strategy):
        self.fee_strategy = fee_strategy

    def place_order(self, order):
        fee = self.fee_strategy.fee(order)
        # apply fee, validate, route order...
        return fee


stock_broker = BrokerageService(FlatFee(5.00))
option_broker = BrokerageService(PercentageFee(0.65)


stock_broker.place_order("BUY AAPL")
option_broker.place_order("BUY AAPL CALL")
```

Duke

Design Patterns: Disadvantages

- Some patterns come with negative consequences (i.e. object proliferation, performance hits, additional layers)
- Consequences are subjective depending on concrete scenarios
- Patterns are subject to different interpretations, misinterpretations, and philosophies
- Patterns can be overused and abused → Anti-Patterns

Design Patterns Support SOLID Principles

Patterns aren't just recipes — they embody design principles

Single Responsibility Principle (SRP)

- **Strategy Pattern** → Separates algorithm logic from context
- **Observer Pattern** → Separates notification from business logic
- **Factory Pattern** → Separates object creation from object usage

Open/Closed Principle (OCP)

- **Strategy Pattern** → Add new algorithms without changing context
- **Factory Pattern** → Add new product types without changing client code
- **Decorator Pattern** → Add behavior without modifying original class

Liskov Substitution Principle (LSP)

- **Strategy Pattern** → All strategies must honor the same contract
- **Factory Pattern** → All products must be interchangeable through common interface
- **Adapter Pattern** → Can help fix LSP violations by wrapping incompatible types

Interface Segregation Principle (ISP)

- **Adapter Pattern** → Clients see only the interface they need
- **Facade Pattern** → Provides focused, narrow interfaces to subsystems

Dependency Inversion Principle (DIP)

- **Factory Pattern** → Clients depend on abstractions, not concrete classes
- **Strategy Pattern** → Context depends on Strategy interface, not implementations
- **Observer Pattern** → Subject and observers depend on abstract roles

Patterns in this Lecture: Quick Reference

- When creating objects: Factory, Singleton
- When interfacing with legacy code: Adapter
- When simplifying subsystems: Facade
- When traversing collections: Iterator
- When reacting to events: Observer
- When swapping algorithms: Strategy

Remember: Patterns are tools, not rules.

Use judgment. Start simple. Add patterns when needed.

Creation Patterns

- **Abstract Factory**: Creates an instance of several families of classes
- **Builder**: Separates object construction from its representation
- **Factory Method**: Creates an instance of several derived classes
- **Object Pool**: Avoid expensive acquisition and release of resources by recycling objects that are no longer in use
- **Prototype**: A fully initialized instance to be copied or cloned
- **Singleton** A class of which only a single instance can exist

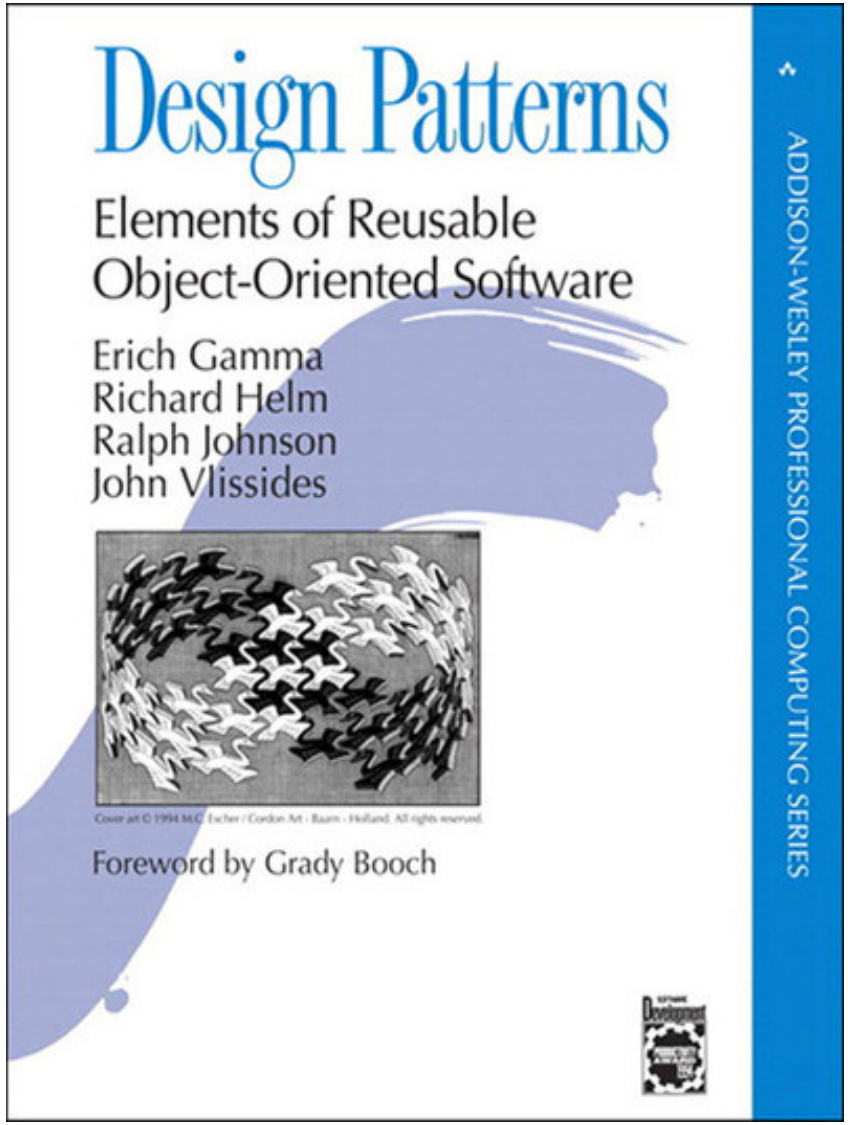
Structural Patterns

- **Adapter**: Match interfaces of different classes
- **Bridge**: Separates an object's interface from its implementation
- **Composite**: A tree structure of simple and composite objects
- **Decorator**: Add responsibilities to objects dynamically
- **Facade**: A single class that represents an entire subsystem
- **Flyweight**: A fine-grained instance used for efficient sharing
- **Private Class Data**: Restricts accessor/mutator access
- **Proxy**: An object representing another object

Behavioral Patterns

- **Chain of responsibility**: A way of passing a request between a chain of objects
- **Command**: Encapsulate a command request as an object
- **Interpreter**: A way to include language elements in a program
- **Iterator**: Sequentially access the elements of a collection
- **Mediator**: Defines simplified communication between classes
- **Memento**: Capture and restore an object's internal state
- **Null Object**: Designed to act as a default value of an object
- **Observer**: A way of notifying change to a number of classes
- **State**: Alter an object's behavior when its state changes
- **Strategy**: Encapsulates an algorithm inside a class
- **Template method**: Defer the exact steps of an algorithm to a subclass
- **Visitor**: Defines a new operation to a class without change

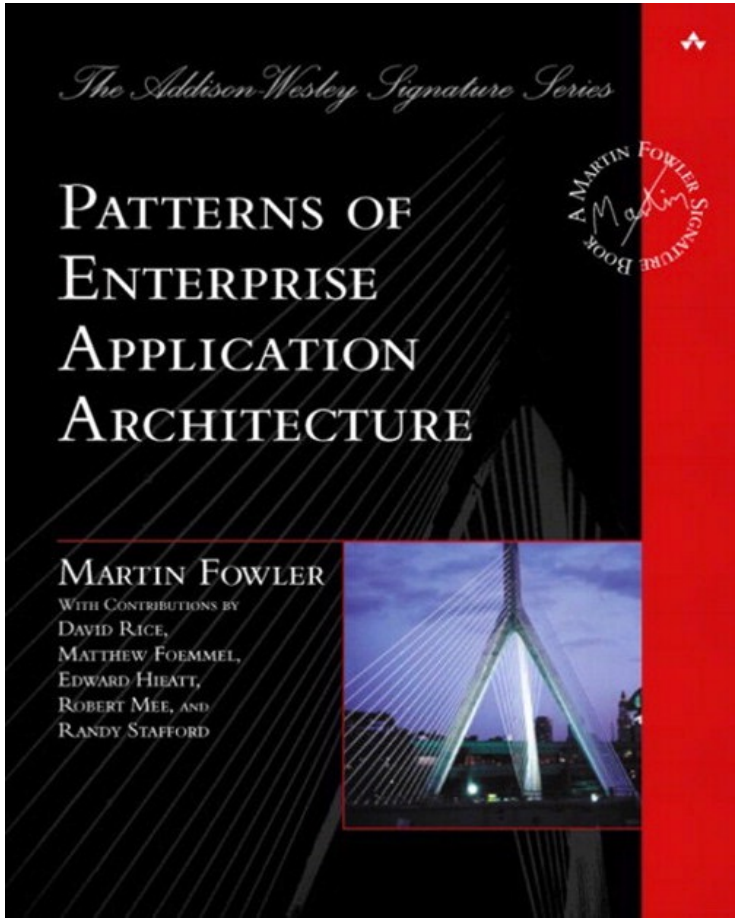
For Reference



https://learning.oreilly.com/library/view/design-patterns-elements/0201633612/

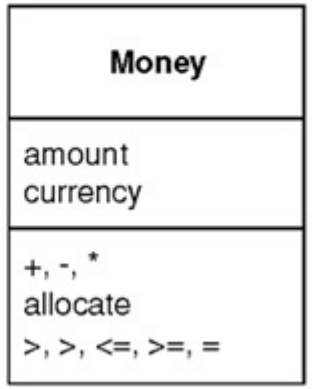
Duke

Patterns: Not just for Design



Money

Represents a monetary value.



Book: https://learning.oreilly.com/library/view/patterns-of-enterprise/0321127420
Money: https://learning.oreilly.com/library/view/patterns-of-enterprise/0321127420/ch18.xhtml#ch18lev1sec7

Duke